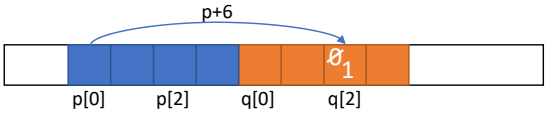

Fig. 1. In a flat memory model, storing a 1 into `p[6]` can overwrite the `0` in `q[2]`

into objects can be constructed using pointer arithmetic, and pointers may be converted to integers and integers to pointers. However, despite their low-level character, the C and C++ memory models also incorporate higher-level features. For example, the compiler is permitted to assume (with some restrictions) that a pointer to one allocated object is not used as the basis for creating a pointer to another object. The C and C++ standards committees intend for the languages to provide low-level memory access when necessary, but otherwise retain the ability to heavily optimize code as if it were high-level. This design point is not an easy one. The tension between the low-level and high-level language features is significant and causes problems for both compiler and application developers.

This paper is about sequential memory models for compiler intermediate representations (IRs). The IR level is important because it is where high-level optimizations happen. Both GCC and LLVM have memory models that are informally specified and that contain problems that lead to end-to-end miscompilation of low-level programs. Appendix A shows a C program that is miscompiled by both GCC and LLVM. However, the problems that we are attacking are broader than C and C++: Appendix B shows a low-level (but safe) Rust function that is miscompiled by LLVM. The culprit is a new bug we found in LLVM's global value numbering (GVN) optimization. GVN propagates equalities of pointers (as well as of integers) from branch conditions, replacing pointers with value-equal ones. This, however, can change the behavior of a program, since pointers that compare equal are not necessarily equivalent. The miscompilation of the C example can be traced back to a second bug we found where LLVM incorrectly assumes that `(int*)(intptr_t)p` is equal to `p`.

Fixing these miscompilations within the current IR semantics would be possible, but would necessitate disabling useful optimizations. Our main contribution, which builds on insights developed by [Kang et al. 2015], is a new, formalized IR memory model for LLVM that departs from the current design in two ways. First, it uses *deferred bounds checking* to relax restrictions on the creation of out-of-bounds pointers in such a way that useful code motion optimizations can be performed soundly. Second, it uses *twin allocation*, which formalizes the idea that the value of a pointer has to be observed directly, it cannot be guessed. Twin allocation supports aggressive optimization of LLVM-based languages in the presence of low-level code such as integer-to-pointer casts. We have adapted LLVM to the new semantics in order to show that it does not require major changes to the compiler and it also does not degrade the performance of generated code.

2 BACKGROUND: MEMORY MODELS FOR INTERMEDIATE REPRESENTATIONS

In this section we describe the design space for low-level sequential memory models and explain that existing designs are inadequate for managing the tension between low-level memory access and high-level optimizations. Then, in Section 3, we describe the basis for the new memory model for LLVM IR that we will formalize in Section 4. Our examples are written in a C-like syntax to make them easier to read, even though the scope of the paper is to specify a memory model for compiler IRs rather than for C.

2.1 Flat Memory Models

The two main questions a memory model needs to answer are (1) what is the return value of a load instruction, and (2) under what conditions is a memory-accessing instruction well-defined. A